

A Mini Project Report

On

STATISTICAL MACHINE TRANSLATION (ENGLISH TO HINDI)

In Subject: **Natural Language Processing**

by

Yash Akotkar	(22010814)
Rajdeep Chaurasia	(22010948)
Netal Daga	(22010244)
Samarth Ghule	(22010468)



Department of Artificial Intelligence and Data Science

VIIT

2022-2023

Guided By:

Prof. Suvarna Bhagwat

Contents

Sr. No.	Topic		Page No.
Chapter-1	Introduction		
	1.1	Introduction	03
	1.2	Requirements	03
	1.3	Problem Statement	04
	1.4	Proposed work	04
Chapter-2	Methodology		
	2.1	Approach	04
	2.2	Platform and Technology	14
	2.3	Outcomes & Working	14
	2.4	Future Work	15
Chapter-3	Conclusion		16
	References		16

1.1 Introduction

- Statistical machine translation, known as SMT or StatMT, is an approach to machine translation which yields the most probable output (translation) of each element that makes up a sentence. It is based on the use of statistical models that analyze and search for relationships between two texts with the same content: one in the source language and the other in the target language.
- SMT uses information theory as its main basis; a study of the storage, processing, extraction, and use of information using statistics, computer science, information engineering, electrical engineering, and statistical mechanics.
- A text is translated based on the probability that a string of words in the target language is the translation of the string of words in the source language.
- This probabilistic distribution model has been approached from different perspectives. The most widely implemented is the **Bayes' Theorem: $p(e|f) \propto p(e|f)p(e)$** , this theorem splits the model into two subproblems. The best translation is obtained by choosing the outcome with the highest probability.
- The different kinds of SMT are as follows:
 - Word-based translation: In this case, the basic translation unit is a word in the source language. In other words, it is a model that translates word for word. However, due to idioms, morphology, and compound words, the number of words in the target sentence is usually different to the source.
 - Phrase-based translation: This type of machine translation technology translates complete word sequences and seeks to decrease the restrictions of word-based SMT. These sequences are called phrases or blocks. However, these phrases are not based on linguistic structures but on statistical methods, so as not to reduce the quality of the translation.
 - Syntax-based translation: In this type of machine translation technology, the SMT model doesn't use the individual words or phrases, but rather translates syntactic units. This means that it translates by analyzing sentences or expressions.

1.2 Requirements

- Language Used: Python
- Library Used
 - Numpy
 - Pandas
 - Keras
 - String
 - NLTK
 - Re
 - Input, LSTM, Dense, Time Distributed, Embedding, Bidirectional
- Software Used: Visual Studio Code

1.3 Problem Statement

Implement Statistical Machine Translation for English to Hindi language translation using different NLP libraries. Predict the hindi sentence translation of given English sentence as the input.

1.4 Proposed Work

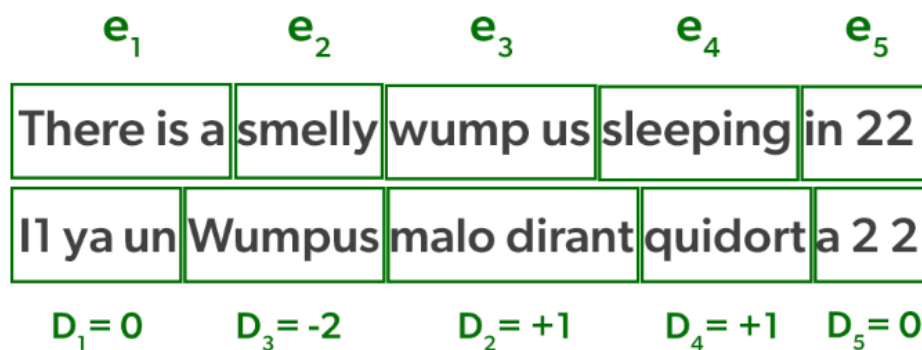
It just requires data in the form of example translations from which a translation model may be learned. We determine the string of words(f^*) that maximizes:

$$f^* = \operatorname{argmax}_f P(f | e) = \operatorname{argmax}_f P(e | f)P(f)$$

The target language model for Hindi is " $P(f)$ ", which indicates how likely a particular sentence is in Hindi. The translation model, $P(e | f)$, indicates how likely an English sentence is to be translated into a particular Hindi sentence. Similarly, $P(f | e)$ is an English-to-Hindi translation model.

Given an English sentence (e), finding a Hindi translation $f()$ is a three-step process:

1. Divide the English sentence into $e_1, \dots, e_n$.
2. Choose a French phrase f_i for each phrase e_i . For the phrasal probability that f_i is a translation of e_i , we use the notation $P(f_i | e_i)$.
3. Pick a combination of the terms $f_1, \dots, f_n$. This permutation will be specified in a style that appears difficult but is supposed to have a simple probability distribution: We pick a distortion d_i for each f_i , which is the number of words that phrase f_i has moved with regard to f_{i-1} ; positive for travelling to the right, negative for moving to the left, and zero if f_i follows f_{i-1} directly.



2.1 Approach

Operational phases of statistical machine translation:

- **Elaborating the parallel text steps:**
 - Choice: two texts or documents with the same content are chosen, one in the source language and the other in the target language. The larger the volume of text, the higher the quality of the final translation.
 - Extraction: sections of the content are extracted from the source language text and the corresponding section in the target language.
 - Separation: each section is broken down into sentences.
 - Preparation: entries are prepared for the system.

- Alignment: each sentence in one language is mapped to the corresponding sentence in the other language.

- **Modeling:**

- Translation modeling: determines the set of possible translations for each of the sentences.
- Language modeling: determines the flow of each sentence. This model is the one that assigns the highest probability to the sentence that uses the most natural language.
- Searching: this is the process in which the system navigates through all the aligned sentences in order to find the most likely translation for a given sentence.

- **Estimating and Refining:**

The estimation and refining phase minimizes any possible errors for a higher quality result. Grammatical connectors and heuristic algorithms are used for this purpose.

```
import numpy as np
import pandas as pd
import keras
from keras.models import Model
from keras.layers import Input, LSTM, Dense, TimeDistributed, Embedding, Bidirectional
from keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import pad_sequences
from string import digits
import nltk
import re
import string
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_columns', 500)
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', -1)
```

Using TensorFlow backend.

```
lines = pd.read_csv('/kaggle/input/english-to-hindi-parallel-dataset/newdata.csv')
lines = lines[:30000]
lines.head()
```

Unnamed: 0	english_sentence	hindi_sentence
0	0 politicians do not have permission to do what needs to be done.	राजनीतिज्ञों के पास जो कार्य करना चाहिए, वह करने कि अनुमति नहीं है।
1	1 I'd like to tell you about one such child,	मई आपको ऐसे ही एक बच्चे के बारे में बताना चाहूंगी,
2	2 This percentage is even greater than the percentage in India.	यह प्रतिशत भारत में हिन्दुओं प्रतिशत से अधिक है।
3	3 what we really mean is that they're bad at not paying attention.	हम ये नहीं कहना चाहते कि वो ध्यान नहीं दे पाते
4	4 .The ending portion of these Vedas is called Upanishad.	इन्हीं वेदों का अंतिम भाग उपनिषद कहलाता है।

```

# Lowercase all characters
lines['english_sentence']=lines['english_sentence'].apply(lambda x: str(x))
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: str(x))
lines['english_sentence']=lines['english_sentence'].apply(lambda x: x.lower())
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: x.lower())

# Remove quotes
lines['english_sentence']=lines['english_sentence'].apply(lambda x: re.sub('"', '', x))
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: re.sub('"', '', x))

exclude = set(string.punctuation) # Set of all special characters
# Remove all the special characters
lines['english_sentence']=lines['english_sentence'].apply(lambda x: ''.join(ch for ch in x if ch not in exclude))
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: ''.join(ch for ch in x if ch not in exclude))

```

```

# Remove all numbers from text
remove_digits = str.maketrans('', '', digits)
lines['english_sentence']=lines['english_sentence'].apply(lambda x: x.translate(remove_digits))
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: x.translate(remove_digits))

lines['hindi_sentence'] = lines['hindi_sentence'].apply(lambda x: re.sub("[२३०८१५७९४६]", "", x))

# Remove extra spaces
lines['english_sentence']=lines['english_sentence'].apply(lambda x: x.strip())
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: x.strip())
lines['english_sentence']=lines['english_sentence'].apply(lambda x: re.sub(" +", " ", x))
lines['hindi_sentence']=lines['hindi_sentence'].apply(lambda x: re.sub(" +", " ", x))

# Add start and end tokens to target sequences
lines['hindi_sentence'] = lines['hindi_sentence'].apply(lambda x : 'START_ ' + x + ' _END')

```

```

### Get English and Hindi Vocabulary
all_eng_words=set()
for eng in lines['english_sentence']:
    for word in eng.split():
        if word not in all_eng_words:
            all_eng_words.add(word)

all_hindi_words=set()
for hin in lines['hindi_sentence']:
    for word in hin.split():
        if word not in all_hindi_words:
            all_hindi_words.add(word)

```

```
lines['length_eng_sentence']=lines['english_sentence'].apply(lambda x:len(x.split(" ")))
lines['length_hin_sentence']=lines['hindi_sentence'].apply(lambda x:len(x.split(" ")))
```

```
lines.head()
lines[lines['length_eng_sentence']>30].shape
```

```
(2919, 5)
```

```
lines=lines[lines['length_eng_sentence']<=20]
lines=lines[lines['length_hin_sentence']<=20]
```

```
print("maximum length of Hindi Sentence ",max(lines['length_hin_sentence']))
print("maximum length of English Sentence ",max(lines['length_eng_sentence']))
```

```
maximum length of Hindi Sentence  20
maximum length of English Sentence  20
```

```
max_length_src=max(lines['length_hin_sentence'])
max_length_tar=max(lines['length_eng_sentence'])
```

```
input_words = sorted(list(all_eng_words))
target_words = sorted(list(all_hindi_words))
num_encoder_tokens = len(all_eng_words)
num_decoder_tokens = len(all_hindi_words)
num_encoder_tokens, num_decoder_tokens
```

```
(33708, 40802)
```

```
num_decoder_tokens += 1
```

```
input_token_index = dict([(word, i+1) for i, word in enumerate(input_words)])
target_token_index = dict([(word, i+1) for i, word in enumerate(target_words)])
```

These two lines of code create dictionaries that map each unique word in the input and target vocabularies to a unique integer index, which will be used to represent the words as one-hot encoded vectors during training and inference. Here what each line does:

- **input_token_index = dict([(word, i+1) for i, word in enumerate(input_words)])**: creates a dictionary input_token_index where each unique word in input_words (a list of all words in the input vocabulary) is mapped to a unique integer index. The index starts at 1 (not 0) because 0 will

be used as a padding token. The `enumerate()` function generates a sequence of (index, word) pairs for each word in `input_words`, which are then converted to a dictionary using the `dict()` function.

- **`target_token_index = dict([(word, i+1) for i, word in enumerate(target_words)])`**: creates a similar dictionary `target_token_index` for the target vocabulary.

Overall, these lines of code create two lookup tables that allow the model to efficiently convert between words and their integer representations, which is necessary for training and inference.

```
reverse_input_char_index = dict((i, word) for word, i in input_token_index.items())
reverse_target_char_index = dict((i, word) for word, i in target_token_index.items())

lines.head(10)
```

These two lines of code create reverse dictionaries that map each unique integer index in the input and target vocabularies back to the corresponding word. Here's what each line does:

- **`reverse_input_char_index = dict((i, word) for word, i in input_token_index.items())`**: creates a dictionary `reverse_input_char_index` where each integer index in `input_token_index` is mapped to the corresponding word. The `items()` method of the `input_token_index` dictionary returns a sequence of (word, index) pairs, which are then reversed and converted to a dictionary using the `dict()` function.
- **`reverse_target_char_index = dict((i, word) for word, i in target_token_index.items())`**: creates a similar dictionary `reverse_target_char_index` for the target vocabulary.

Overall, these lines of code create two lookup tables that allow the model to efficiently convert between integer representations and their corresponding words, which is useful for inspecting the model's outputs and generating text during inference.

...	Unnamed: 0	english_sentence	hindi_sentence	length_eng_sentence	length_hin_sentence
0	0	politicians do not have permission to do what needs to be done	START_ राजनीतिज्ञों के पास जो कार्य करना चाहिए वह करने कि अनुमति नहीं है _END	12	15
1	1	id like to tell you about one such child	START_ मई आपको ऐसे ही एक बच्चे के बारे में बताना चाहूंगी _END	9	13
2	2	this percentage is even greater than the percentage in india	START_ यह प्रतिशत भारत में हिन्दुओं प्रतिशत से अधिक है। _END	10	11
3	3	what we really mean is that they're bad at not paying attention	START_ हम ये नहीं कहना चाहते कि वो ध्यान नहीं दे पाते _END	12	13
4	4	the ending portion of these vedas is called upanishad	START_ इन्हीं वेदों का अंतिम भाग उपनिषद कहलाता है। _END	9	10
6	6	in this lies the circumstances of people before you	START_ इसमें तुमसे पूर्व गुज़रे हुए लोगों के हालात हैं। _END	9	11
7	7	and who are we to say even that they are wrong	START_ और हम होते कौन हैं यह कहने भी वाले कि वे गलत हैं _END	11	15
10	10	please ensure that you use the appropriate form	START_ कृपया यह सुनिश्चित कर लें कि आप सही फॉर्म का प्रयोग कर रहे हैं _END	8	16
11	11	category religious text	START_ श्रेणीधर्मग्रन्थ _END	3	3
12	12	this period summarily is pepped up with devotion	START_ यह काल समग्रतः भक्ति भावना से ओतप्रोत काल है। _END	8	11


```

from sklearn.model_selection import train_test_split
X, y = lines['english_sentence'], lines['hindi_sentence']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42)
X_train.shape, X_test.shape

[20]
... ((16007,), (4002,))

X_train

[21]
... 21471 the future development of the mini steel industry will depend on the availability of sponge iron
2807 and god“ he said to the poor man ”is answering your plea
1577 deep shade of white mausoleum could clearly be seen in the lake
7990 hindu philosophy of unity
20023 to stand up and join in
...
16973 hindu people really do not worship stones and iron as some people explain
17972 he is self originated and maker of world
8002 during this period considerable political consciousness developed in the country
1303 badruddin himself had to learn all these languages before his english education began
23756 you really are considered a bit of an antique
Name: english_sentence, Length: 16007, dtype: object

```

```

def generate_batch(X = X_train, y = y_train, batch_size = 128):
    ''' Generate a batch of data '''
    while True:
        for j in range(0, len(X), batch_size):
            encoder_input_data = np.zeros((batch_size, max_length_src), dtype='float32')
            decoder_input_data = np.zeros((batch_size, max_length_tar), dtype='float32')
            decoder_target_data = np.zeros((batch_size, max_length_tar, num_decoder_tokens), dtype='float32')
            for i, (input_text, target_text) in enumerate(zip(X[j:j+batch_size], y[j:j+batch_size])):
                for t, word in enumerate(input_text.split()):
                    encoder_input_data[i, t] = input_token_index[word] # encoder input seq
                for t, word in enumerate(target_text.split()):
                    if t < len(target_text.split()) - 1:
                        decoder_input_data[i, t] = target_token_index[word] # decoder input seq
                    if t > 0:
                        # decoder target sequence (one hot encoded)
                        # does not include the START_ token
                        # Offset by one timestep
                        decoder_target_data[i, t - 1, target_token_index[word]] = 1.
            yield([encoder_input_data, decoder_input_data], decoder_target_data)

```

This function generates a batch of training data for the model during each iteration of training. The function uses a Python generator to lazily generate data on-the-fly, rather than loading all the data into memory at once. This can be more memory-efficient for large datasets. Here's how the function works:

- **while True:** loop ensures that the generator keeps generating batches indefinitely, so that the model can continue training on new data.
- **The outer for j in range(0, len(X), batch_size):** loop iterates over the input data in chunks of batch_size examples at a time. Each chunk of examples is processed together in a single batch.
- **Inside the loop,** the function initializes three numpy arrays encoder_input_data, decoder_input_data, and decoder_target_data, each with dimensions (batch_size, max_length_src), (batch_size, max_length_tar), and (batch_size, max_length_tar, num_decoder_tokens) respectively. These arrays will be filled with one-hot encoded representations of the input, decoder input, and decoder output sequences.

- The nested inner loops iterate over each example in the current batch and tokenize the input and target sequences into individual words using `.split()`. Then, each word is looked up in the corresponding token index dictionary (`input_token_index` or `target_token_index`) to get its integer representation.
- The integer representations are then stored in the appropriate numpy array (`encoder_input_data`, `decoder_input_data`, or `decoder_target_data`) at the appropriate time step (represented by the `t` variable).
- **function yields** a tuple of (`encoder_input_data`, `decoder_input_data`) and `decoder_target_data`, which can be fed into the model during training using the `fit_generator()` method. This will generate one batch of training data at a time, allowing the model to train on the entire dataset over multiple epochs.

```
#this code sets up an LSTM-based encoder for a sequence-to-sequence model, which takes an input sequence of variable length,
#processes it with an embedding layer and an LSTM layer, and outputs the final hidden and cell states of the LSTM.

latent_dim = 300 #sets the dimensionality of the hidden state of the LSTM layer.
# Encoder
#is a placeholder for the input sequence, with shape (None,). This allows the model to accept input sequences of variable length.
encoder_inputs = Input(shape=(None,))

#The num_encoder_tokens parameter specifies the number of unique tokens in the input vocabulary,
#and mask_zero=True indicates that any padding tokens in the input sequence should be masked out.
enc_emb = Embedding(num_encoder_tokens, latent_dim, mask_zero = True)(encoder_inputs)

#creates an LSTM layer with latent_dim units, and return_state=True instructs the
#layer to return the final hidden state and cell state of the LSTM after processing the input sequence.
encoder_lstm = LSTM(latent_dim, return_state=True)

encoder_outputs, state_h, state_c = encoder_lstm(enc_emb)

# We discard `encoder_outputs` and only keep the states.
# creates a list containing the final hidden state and cell state of the encoder LSTM,
# which will be used as the initial state for the decoder LSTM in the sequence-to-sequence model.
encoder_states = [state_h, state_c]
```

Python

This code snippet defines an encoder for a sequence-to-sequence model using a recurrent neural network (RNN) architecture, specifically a Long Short-Term Memory (LSTM) network. Here's a breakdown of what the code is doing:

- **latent_dim = 300** sets the dimensionality of the hidden state of the LSTM layer.
- **encoder_inputs** is a placeholder for the input sequence, with shape `(None,)`. This allows the model to accept input sequences of variable length.
- **Embedding(num_encoder_tokens, latent_dim, mask_zero = True)** creates an embedding layer for the input sequence. The `num_encoder_tokens` parameter specifies the number of unique tokens in the input vocabulary, and `mask_zero=True` indicates that any padding tokens in the input sequence should be masked out.
- **encoder_lstm = LSTM(latent_dim, return_state=True)** creates an LSTM layer with `latent_dim` units, and `return_state=True` instructs the layer to return the final hidden state and cell state of the LSTM after processing the input sequence.
- **encoder_outputs, state_h, state_c = encoder_lstm(enc_emb)** applies the LSTM layer to the input sequence (`enc_emb`) and returns the final hidden state (`state_h`) and cell state (`state_c`) of the LSTM, as well as the entire sequence of hidden states (`encoder_outputs`). Note that `encoder_outputs` is not used further in this code snippet.

- **encoder_states = [state_h, state_c]** creates a list containing the final hidden state and cell state of the encoder LSTM, which will be used as the initial state for the decoder LSTM in the sequence-to-sequence model.

Overall, this code sets up an LSTM-based encoder for a sequence-to-sequence model, which takes an input sequence of variable length, processes it with an embedding layer and an LSTM layer, and outputs the final hidden and cell states of the LSTM.

```
# Set up the decoder, using `encoder_states` as initial state.
decoder_inputs = Input(shape=(None,))
dec_emb_layer = Embedding(num_decoder_tokens, latent_dim, mask_zero = True)
dec_emb = dec_emb_layer(decoder_inputs)
# We set up our decoder to return full output sequences,
# and to return internal states as well. We don't use the
# return states in the training model, but we will use them in inference.
decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True)
decoder_outputs, _, _ = decoder_lstm(dec_emb,
                                     initial_state=encoder_states)
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_outputs)

# Define the model that will turn
# `encoder_input_data` & `decoder_input_data` into `decoder_target_data`
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

This code snippet defines a sequence-to-sequence model with an LSTM-based decoder. Here's a breakdown of what the code is doing:

- **decoder_inputs** is a placeholder for the decoder input sequence, with shape (None,).
- **dec_emb_layer** = `Embedding(num_decoder_tokens, latent_dim, mask_zero = True)` creates an embedding layer for the decoder input sequence, similar to the one used for the encoder input sequence.
- **dec_emb** = `dec_emb_layer(decoder_inputs)` applies the embedding layer to the decoder input sequence.
- **decoder_lstm** = `LSTM(latent_dim, return_sequences=True, return_state=True)` creates an LSTM layer for the decoder with `latent_dim` units, which returns the full output sequence (`return_sequences=True`) as well as the final hidden and cell states (`return_state=True`).
- **decoder_outputs, _, _** = `decoder_lstm(dec_emb, initial_state=encoder_states)` applies the decoder LSTM to the embedded decoder input sequence (`dec_emb`) with the initial hidden and cell states set to the final states of the encoder LSTM (`encoder_states`). Note that the second and third return values of the LSTM (the cell state and the final state) are ignored with `_`.
- **decoder_dense** = `Dense(num_decoder_tokens, activation='softmax')` creates a dense layer with a softmax activation function, which maps the output of the decoder LSTM to a probability distribution over the vocabulary of the output language.

- **decoder_outputs** = decoder_dense(decoder_outputs) applies the dense layer to the output of the decoder LSTM to obtain the final output sequence of the decoder.
- **model** = Model([encoder_inputs, decoder_inputs], decoder_outputs) defines the full sequence-to-sequence model, which takes as input both the encoder input sequence and the decoder input sequence, and produces as output the decoder output sequence.

Overall, this code sets up a sequence-to-sequence model with an LSTM-based encoder and decoder, which takes as input an encoder input sequence and a decoder input sequence, processes them with embedding layers and LSTM layers, and produces as output a decoder output sequence.

```
model.summary()
train_samples = len(X_train)
val_samples = len(X_test)
batch_size = 64
epochs = 100
```

Model: "model_1"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	(None, None)	0	
input_2 (InputLayer)	(None, None)	0	
embedding_1 (Embedding)	(None, None, 300)	10112400	input_1[0][0]
embedding_2 (Embedding)	(None, None, 300)	12240900	input_2[0][0]
lstm_1 (LSTM)	[(None, 300), (None, 721200		embedding_1[0][0]
lstm_2 (LSTM)	[(None, None, 300), 721200		embedding_2[0][0] lstm_1[0][1] lstm_1[0][2]
dense_1 (Dense)	(None, None, 40803)	12281703	lstm_2[0][0]
Total params: 36,077,403			
Trainable params: 36,077,403			
Non-trainable params: 0			

```
model.fit_generator(generator = generate_batch(X_train, y_train, batch_size = batch_size),
                    steps_per_epoch = train_samples//batch_size,
                    epochs=5,
                    validation_data = generate_batch(X_test, y_test, batch_size = batch_size),
                    validation_steps = val_samples//batch_size)
```

```

/opt/conda/lib/python3.6/site-packages/tensorflow_core/python/framework/indexed_slices.py:433: UserWarning: Converting sparse IndexedSlices
"Converting sparse IndexedSlices to a dense Tensor of unknown shape. "
Epoch 1/5
250/250 [=====] - 168s 673ms/step - loss: 3.7389 - accuracy: 0.1133 - val_loss: 3.6945 - val_accuracy: 0.1199
Epoch 2/5
250/250 [=====] - 165s 659ms/step - loss: 3.2724 - accuracy: 0.1368 - val_loss: 3.0422 - val_accuracy: 0.1436
Epoch 3/5
250/250 [=====] - 165s 660ms/step - loss: 3.0760 - accuracy: 0.1588 - val_loss: 3.4962 - val_accuracy: 0.1607
Epoch 4/5
250/250 [=====] - 165s 659ms/step - loss: 2.9010 - accuracy: 0.1814 - val_loss: 3.1662 - val_accuracy: 0.1733
Epoch 5/5
250/250 [=====] - 165s 660ms/step - loss: 2.7468 - accuracy: 0.1997 - val_loss: 3.1484 - val_accuracy: 0.1823

<keras.callbacks.callbacks.History at 0x7fcf5005ef98>

model.save('eng-to-hindi.h5')

```

```

train_gen = generate_batch(X_train, y_train, batch_size = 1)
k=-1

# Encode the input sequence to get the "thought vectors"
encoder_model = Model(encoder_inputs, encoder_states)

# Decoder setup
# Below tensors will hold the states of the previous time step
decoder_state_input_h = Input(shape=(latent_dim,))
decoder_state_input_c = Input(shape=(latent_dim,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]

dec_emb2= dec_emb_layer(decoder_inputs) # Get the embeddings of the decoder sequence

# To predict the next word in the sequence, set the initial states to the states from the previous time step
decoder_outputs2, state_h2, state_c2 = decoder_lstm(dec_emb2, initial_state=decoder_states_inputs)
decoder_states2 = [state_h2, state_c2]
decoder_outputs2 = decoder_dense(decoder_outputs2) # A dense softmax layer to generate prob dist. over the target vocabulary

# Final decoder model
decoder_model = Model(
    [decoder_inputs] + decoder_states_inputs,
    [decoder_outputs2] + decoder_states2)

```

This code sets up the inference model for the seq2seq model. The inference model is used to generate predictions for new input sequences that were not seen during training.

The encoder model takes the input sequence and generates the hidden state and cell state vectors, which represent the "thought vectors" for the input sequence. These vectors are then used to initialize the decoder model.

The decoder model takes the "thought vectors" and the decoder input sequence and generates the output sequence. In this case, the decoder input sequence is just a start-of-sequence token followed by an empty sequence. At each time step, the model generates a probability distribution over the target vocabulary using a dense softmax layer. The word with the highest probability is chosen as the output at each time step.

The decoder states (i.e., the hidden state and cell state vectors) are passed back to the model at the next time step as inputs, allowing the model to generate the next word in the sequence. This process is repeated until an end-of-sequence token is generated or the maximum sequence length is reached.

```

def decode_sequence(input_seq):
    states_value = encoder_model.predict(input_seq) # Encode the input as state vectors.
    # Generate empty target sequence of length 1.
    target_seq = np.zeros((1,1))
    # Populate the first character of target sequence with the start character.
    target_seq[0, 0] = target_token_index['START_']

    # Sampling loop for a batch of sequences (to simplify, here we assume a batch of size 1).
    stop_condition = False
    decoded_sentence = ''
    while not stop_condition:
        output_tokens, h, c = decoder_model.predict([target_seq] + states_value)

        sampled_token_index = np.argmax(output_tokens[0, -1, :]) # Sample a token
        sampled_char = reverse_target_char_index[sampled_token_index]
        decoded_sentence += ' '+sampled_char

        # Exit condition: either hit max length or find stop character.
        if (sampled_char == '_END' or
            len(decoded_sentence) > 50):
            stop_condition = True

        # Update the target sequence (of length 1).
        target_seq = np.zeros((1,1))
        target_seq[0, 0] = sampled_token_index

        # Update states
        states_value = [h, c]
    return decoded_sentence

```

2.2 Platform & Technology

We have used text editor – Visual Studio Code to implement it. Instead of this there are more options also like eclipse, google colab, jupyter notebook, atom, etc.

2.3 Output & Working

```

k+=1
(input_seq, actual_output), _ = next(train_gen)
decoded_sentence = decode_sequence(input_seq)
print('Input English sentence:', X_train[k:k+1].values[0])
print('Actual Hindi Translation:', y_train[k:k+1].values[0][6:-4])
print('Predicted Hindi Translation:', decoded_sentence[:-4])

```

[33]

... Input English sentence: and god“ he said to the poor man ”is answering your plea
 Actual Hindi Translation: “और भगवान्” उन्होंने गरीब व्यक्ति से कहा “तुम्हारी प्रार्थना सुन रहे हैं
 Predicted Hindi Translation: और यह एक बार एक तरह से एक तरह से नहीं है

```

k+=1
(input_seq, actual_output), _ = next(train_gen)
decoded_sentence = decode_sequence(input_seq)
print('Input English sentence:', X_train[k:k+1].values[0])
print('Actual Hindi Translation:', y_train[k:k+1].values[0][6:-4])
print('Predicted Hindi Translation:', decoded_sentence[:-4])

```

[34]

... Input English sentence: deep shade of white mausoleum could clearly be seen in the lake
 Actual Hindi Translation: श्वेत मकबरे की गहरी छाया को स्पष्ट देखा जा सकता था उस सरोवर में।
 Predicted Hindi Translation: इस प्रकार मैं एक प्रकार मैं एक बार से भी है।

Here's what's happening in each line of code:

- **k+=1:** This increments the value of k by 1. This is used to get the next sentence from the training data.
- **(input_seq, actual_output), _ = next(train_gen):** This generates the next batch of data using the train_gen generator function defined earlier. The batch size is 128 by default. The next() function is used to get the next batch.
- **decoded_sentence = decode_sequence(input_seq):** This calls the decode_sequence() function to generate a Hindi translation for the English sentence in input_seq.
- **print('Input English sentence:', X_train[k:k+1].values[0]):** This prints the English sentence for which the Hindi translation is being generated. The k:k+1 notation is used to get the sentence as a Pandas DataFrame row. The .values[0] part is used to extract the string value from the DataFrame.
- **print('Actual Hindi Translation:', y_train[k:k+1].values[0][6:-4]):** This prints the actual Hindi translation for the English sentence. The [6:-4] notation is used to remove the "START_" and "_END" tokens from the beginning and end of the string.
- **print('Predicted Hindi Translation:', decoded_sentence[:-4]):** This prints the Hindi translation generated by the model. The[:-4] notation is used to remove the "_END" token from the end of the string.

2.4 Future Work

There are several ways in which statistical machine translation projects can be improved to make them better in the future:

1. More data: Increasing the amount of parallel data available for training the machine translation model can lead to better performance, as the model will be able to learn more patterns and relationships between the two languages.
2. Better preprocessing: Preprocessing techniques like data cleaning, normalization, and tokenization can help improve the quality of the data used to train the model.
3. Advanced algorithms: Advanced algorithms like neural machine translation (NMT) have shown significant improvements in translation quality over traditional statistical machine translation approaches. These models have the ability to learn complex patterns and relationships between the input and output languages, leading to better translation quality.
4. Better evaluation metrics: Better evaluation metrics can help to assess the quality of machine translations more accurately. The current evaluation metric BLEU (bilingual evaluation understudy) has limitations and there is ongoing research on finding better metrics that can accurately measure the quality of translations.
5. Domain-specific data: Collecting domain-specific parallel data can improve the translation quality for specific domains like legal or medical translations. This can be achieved by collecting data from specific domains or by fine-tuning the machine translation model on domain-specific data.
6. Human evaluation: Human evaluation is the most reliable and accurate way to evaluate the quality of machine translations. Human evaluation can be used to supplement existing evaluation metrics and provide more accurate assessments of the quality of translations.

Conclusion

- ✓ SMT works by collecting statistics, that is to say, it bases its method on counting repetitions of phrases and words.
- ✓ Although it is partial and may contain errors, SMT translates the text quickly, which allows:
 - Urgent access to data.
 - Easier work for human translators, since they only have to make corrections.
- ✓ Offers more cost-effective translations and a better use of resources, although its quality is not at the level of professional translators.

References

- <https://blog.pangeanic.com/what-is-statistical-machine-translation>
- <https://towardsdatascience.com/machine-translation-b0f0dbcef47c>
- https://www.researchgate.net/publication/307174794_Statistical_Machine_Translation